L2-1.3. Intro to Log Analysis

Intro to Log Analysis

Task 1 - Introduction

Log analysis examines and interprets log event data generated by various sources (devices, applications, and systems) to monitor metrics and identify security incidents. It involves collecting, parsing, and processing log files to turn data into actionable objectives.

Task 2 - Log Analysis Basics

Logs are recorded events or transactions within a system, device, or application. Specifically, these events can be related to application errors, system faults, audited user actions, resource uses, network connections, and more. Each log entry contains relevant details to contextualize the event, such as its timestamp (the date and time it occurred), the source (the system that generated the log), and additional information about the specific log event.

Why Are Logs Important? There are several reasons why collecting logs and adopting an effective log analysis strategy is vital for an organization's ongoing operations. Some of the most common activities include:

- **System Troubleshooting**: Analyzing system errors and warning logs helps IT teams understand and quickly respond to system failures, minimizing downtime, and improving overall system reliability.
- **Cyber Security Incidents:** In the security context, logs are crucial in detecting and responding to security incidents. Firewall logs, intrusion detection system (IDS) logs, and system authentication logs, for example, contain vital information about potential threats and suspicious activities. Performing log analysis helps SOC teams and Security Analysts identify and quickly respond to unauthorized access attempts, malware, data breaches, and other malicious activities.

- **Threat Hunting:** On the proactive side, cyber security teams can use collected logs to actively search for advanced threats that may have evaded traditional security measures. Security Analysts and Threat Hunters can analyze logs to look for unusual patterns, anomalies, and indicators of compromise (IOCs) that might indicate the presence of a threat actor.
- **Compliance:** Organizations must often maintain detailed records of their system's activities for regulatory and compliance purposes. Regular log analysis ensures that organizations can provide accurate reports and demonstrate compliance with regulations such as GDPR, HIPAA or PCI DSS.

Types of Logs

- **Application Logs:** Messages from specific applications, providing insights into their status, errors, warnings, and other operational details.
- **Audit Logs:** Events, actions, and changes occurring within a system or application, providing a history of user activities and system behavior.
- **Security Logs:** Security-related events like logins, permission alterations, firewall activities, and other actions impacting system security.
- **Server Logs:** System logs, event logs, error logs, and access logs, each offering distinct information about server operations.
- **System Logs:** Kernel activities, system errors, boot sequences, and hardware status, aiding in diagnosing system issues.
- **Network Logs:** Communication and activity within a network, capturing information about events, connections, and data transfers.
- **Database Logs:** Activities within a database system, such as queries performed, actions, and updates.
- **Web Server Logs:** Requests processed by web servers, including URLs, source IP addresses, request types, response codes, and more.

Task 3 - Investigation Theory

- **Timeline:** Ordered sequence of events used to understand how an incident happened from start to finish.

- **Timestamp:** Time recorded for each log entry; often normalized so logs from different systems can be compared accurately.
- **Super Timeline:** A merged timeline combining logs from multiple sources (system, network, apps) to give a full picture of activity across an environment.
- **Data Visualization:** Turning log data into graphs, charts, and dashboards to quickly identify trends, spikes, and unusual behavior.
- **Log Monitoring & Alerting:** Continuous observation of logs with automatic alerts when suspicious or predefined activities occur, enabling fast response.
- **External Research & Threat Intelligence:** Using external data (like malicious IPs, domains, hashes) to enrich log analysis and identify known threats or attackers.

Task 4 - Detection Engineering

Common Log File Locations

- **Web Servers:**
  - **Nginx:**
    - Access Logs: `/var/log/nginx/access.log`
    - Error Logs: `/var/log/nginx/error.log`
  - **Apache:**
    - Access Logs: `/var/log/apache2/access.log`
    - Error Logs: `/var/log/apache2/error.log`
- **Databases:**
  - **MySQL:**
    - Error Logs: `/var/log/mysql/error.log`
  - **PostgreSQL:**
    - Error and Activity Logs: `/var/log/postgresql/postgresql-{version}-main.log`
- **Web Applications:**

    - **PHP:**
        - Error Logs: `/var/log/php/error.log`
- **Operating Systems:**
    - **Linux:**
        - General System Logs: `/var/log/syslog`
        - Authentication Logs: `/var/log/auth.log`
- **Firewalls and IDS/IPS:**
    - **iptables:**
        - Firewall Logs: `/var/log/iptables.log`
    - **Snort:**
        - Snort Logs: `/var/log/snort/`

Common Patterns: The specific indicators can vary greatly depending on the source, but some examples of this that can be found in log files include:

- **Abnormal User Behavior:** Activities that deviate from a user's normal behavior and may indicate a security threat.
- **Multiple Failed Logins:** Many failed login attempts in a short time, often a sign of a brute-force attack.
- **Unusual Login Times:** Logins occurring outside a user's normal working hours.
- **Geographic Anomalies:** Logins from unexpected countries or impossible travel locations.
- **Frequent Password Changes:** Repeated password changes that may indicate account takeover attempts.
- **Unusual User-Agent Strings:** Suspicious tools or automated scripts identified through uncommon browser identifiers (e.g., Nmap, Hydra).
- **Behavior Analytics (UBA):** Security tools that use baselines and machine learning to detect unusual user activity and generate alerts.

Common Attack Signatures: Recognizable patterns in logs that indicate malicious activity or exploitation attempts.

- **SQL Injection (SQLi):** Attempts to manipulate database queries using malicious SQL commands, often identified by characters like `'`, `-`, `UNION SELECT`, or functions such as `SLEEP()`.
- **Cross-Site Scripting (XSS):** Injection of malicious JavaScript into web pages, commonly detected through payloads containing `<script>` tags or event handlers like `onerror` and `onclick`.
- **Path Traversal:** Attempts to access unauthorized files by navigating directories using sequences like `../` or encoded variants (`%2E%2F`), often targeting files such as `/etc/passwd`.
- **URL Encoding:** A technique attackers use to disguise malicious payloads by encoding special characters (e.g., `%2E` = `.`, `%2F` = `/`) to evade detection.
- **Log Analysis Goal:** Detect these attack patterns early to identify exploitation attempts and respond before systems are compromised.

Answer the questions below

> What is the default file path to view logs regarding HTTP requests on an Nginx server? /var/log/nginx/access.log

> A log entry containing %2E%2E%2F%2E%2E%2Fproc%2Fself%2Fenviron was identified. What kind of attack might this infer? Path Traversal

Task 5 - Automated vs. Manual Analysis

Analysis Methods

Automated Analysis: Uses tools and AI/ML to process logs, detect patterns, and identify potential threats automatically.

Advantages:

- Saves time by automating log analysis.
- Detects patterns and trends efficiently using AI/ML.

Disadvantages:

- Often requires expensive commercial tools.
- May generate false positives or miss new/unknown threats.

Manual Analysis: Involves human review of logs and security data without relying on automation.

Advantages:

- Low cost and can be performed with basic tools.
- Enables deeper investigation and contextual understanding.
- Reduces reliance on potentially inaccurate automated alerts.

Disadvantages:

- Time-consuming and labor-intensive.
- Large volumes of data can cause important events to be overlooked.

Answer the questions below

> A log file is processed by a tool which returns an output. What form of analysis is this? Automated

> An analyst opens a log file and searches for events. What form of analysis is this? Manual

Task 6 - Log Analysis Tools: Command Line

cat – Displays the entire content of a file in the terminal. Best for small log files.

less – Views large files page by page and allows scrolling without loading the entire file at once.

tail – Shows the last 10 lines of a file by default. Useful for viewing recent log entries.

tail apache.log
tail -f: Monitors logs in real time.
tail -n 20: Shows the last 20 lines.
head: Opposite of tail; shows the first lines of a file.

wc (Word Count) – Provides statistics about a file, including lines, words, and characters.

cut – Extracts specific fields/columns from structured log data using a delimiter.

cut -d ' ' -f 1 apache.log
-f 1 → IP addresses
-f 7 → URLs
-f 9 → HTTP status codes

sort – Sorts data alphabetically or numerically to help identify patterns and trends.

cut -d ' ' -f 1 apache.log | sort -n
-n → Numeric sort
-r → Reverse order

uniq – Removes duplicate entries from sorted data and can count occurrences.

cut -d ' ' -f 1 apache.log | sort | uniq
cut -d ' ' -f 1 apache.log | sort | uniq -c
uniq → Shows unique entries
uniq -c → Shows unique entries with occurrence counts

Common SOC Analyst Workflow
cut -d ' ' -f 1 apache.log | sort | uniq -c | sort -nr

Answer the questions below

> **Use cut on the apache.log file to return only the URLs. What is the flag that is returned in one of the unique entries? c701d43cc5a3acb9b5b04db7f1be94f6**

```
root@ip-10-113-114-25:~/Rooms/introloganalysis/task6# cut -d
' ' -f 7 apache.log
```

> **In the apache.log file, how many total HTTP 200 responses were logged? 52**

```
root@ip-10-113-114-25:~/Rooms/introloganalysis/task6# cut -d
' ' -f 9 apache.log| sort -n -r | uniq -c
```

```
19 404
52 200
```

> **In the apache.log file, which IP address generated the most traffic?**

```
root@ip-10-113-114-25:~/Rooms/introloganalysis/task6# cut -d
' ' -f 1 apache.log | sort -n -r | uniq -c (145.76.33.201)
```

> **What is the complete timestamp of the entry where 110.122.65.76 accessed /login.php?**

```
root@ip-10-113-114-25:~/Rooms/introloganalysis/task6# grep "/
login.php" apache.log | grep "110.122.65.76"
110.122.65.76 - - [31/Jul/2023:12:34:40 +0000] "GET /login.ph
p HTTP/1.1" 200 9876 "Mozilla/5.0 (Windows NT 10.0; Win64; x6
4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.11
0 Safari/537.36"
```

Task 7 - Log Analysis Tools: Regular Expressions

Regex (Regular Expression) is a pattern-matching technique used to search and manipulate text. In cybersecurity, it is important because it helps analysts efficiently detect suspicious activities, extract indicators of compromise, analyze logs, create SIEM detection rules, and automate threat hunting across large volumes of data.

Regex are an invaluable way to define patterns for searching, matching, and manipulating text data. Regular expression patterns are constructed using a combination of special characters that represent matching rules and are supported in many programming languages, text editors, and software.

This regex pattern extracts an IPv4 address from a log entry by matching four groups of 1–3 digits separated by periods. Since the IP address appears at the

beginning of the log, the pattern successfully identifies and highlights it. \b([0-9]{1,3}\.){3}[0-9]{1,3}\b

Why is REGEX Important in Cybersecurity?
Log Analysis: Search large log files for specific patterns such as IP addresses, usernames, error messages, or suspicious requests.
Threat Detection: Identify attack signatures like SQL injection, XSS payloads, or directory traversal attempts.
IOC Hunting: Extract indicators such as IP addresses, domains, URLs, email addresses, and file hashes from logs.
SIEM Rules: Used in tools like Splunk and IBM QRadar to create detection rules and alerts.
Data Validation: Verify whether input matches an expected format (e.g., email addresses or IP addresses).
Incident Response: Quickly find relevant events during investigations.

Logstash and Grok

- **Grok** is a Logstash plugin used to convert unstructured logs into structured, searchable data.
- It uses patterns (`%{SYNTAX:SEMANTIC}`) and **regular expressions (Regex)** to extract information from log entries.
- When built-in patterns are insufficient, custom Regex patterns can be created.
- In the example, a Regex pattern is used to extract **IPv4 addresses** from log messages.
- The extracted IP address is stored in a custom field (`ipv4_address`) before being sent to a SIEM or Elasticsearch.
- Grok helps security analysts normalize and enrich log data, making searching, monitoring, and threat detection easier.

Logstash is an open-source **data collection and processing pipeline** that gathers logs from multiple sources, transforms them, and sends them to a destination such as Elasticsearch, a SIEM, or a database.

It is a core component of the **Elastic Stack (ELK Stack)**:

- **Elasticsearch** → Stores and indexes data

- **Logstash** → Collects, parses, and transforms data
- **Kibana** → Visualizes data

How Logstash Works

1. **Input** – Collects data from sources such as log files, syslog servers, applications, or network devices.
2. **Filter** – Processes and enriches data using plugins like **Grok**, mutate, date, and geoip.
3. **Output** – Sends processed data to destinations like Elasticsearch, SIEMs, databases, or files.

Why is Logstash Important in Cybersecurity?

- Centralizes logs from multiple systems.
- Parses unstructured logs into searchable fields.
- Extracts important information such as IP addresses, usernames, URLs, and status codes.
- Enriches logs with additional context (e.g., geolocation).
- Feeds normalized data into SIEM platforms for threat detection and monitoring.

Logstash is a data processing pipeline used to collect, parse, transform, and forward logs from multiple sources. In cybersecurity, it helps centralize and normalize log data, making it easier for SIEM tools to search, correlate, and detect security threats.

Grok is a Logstash plugin that uses predefined patterns and regular expressions to parse unstructured logs into structured fields. It is widely used in cybersecurity to extract useful information such as IP addresses, usernames, URLs, and timestamps, making logs easier to search, analyze, and use in SIEM platforms.

Answer the questions below

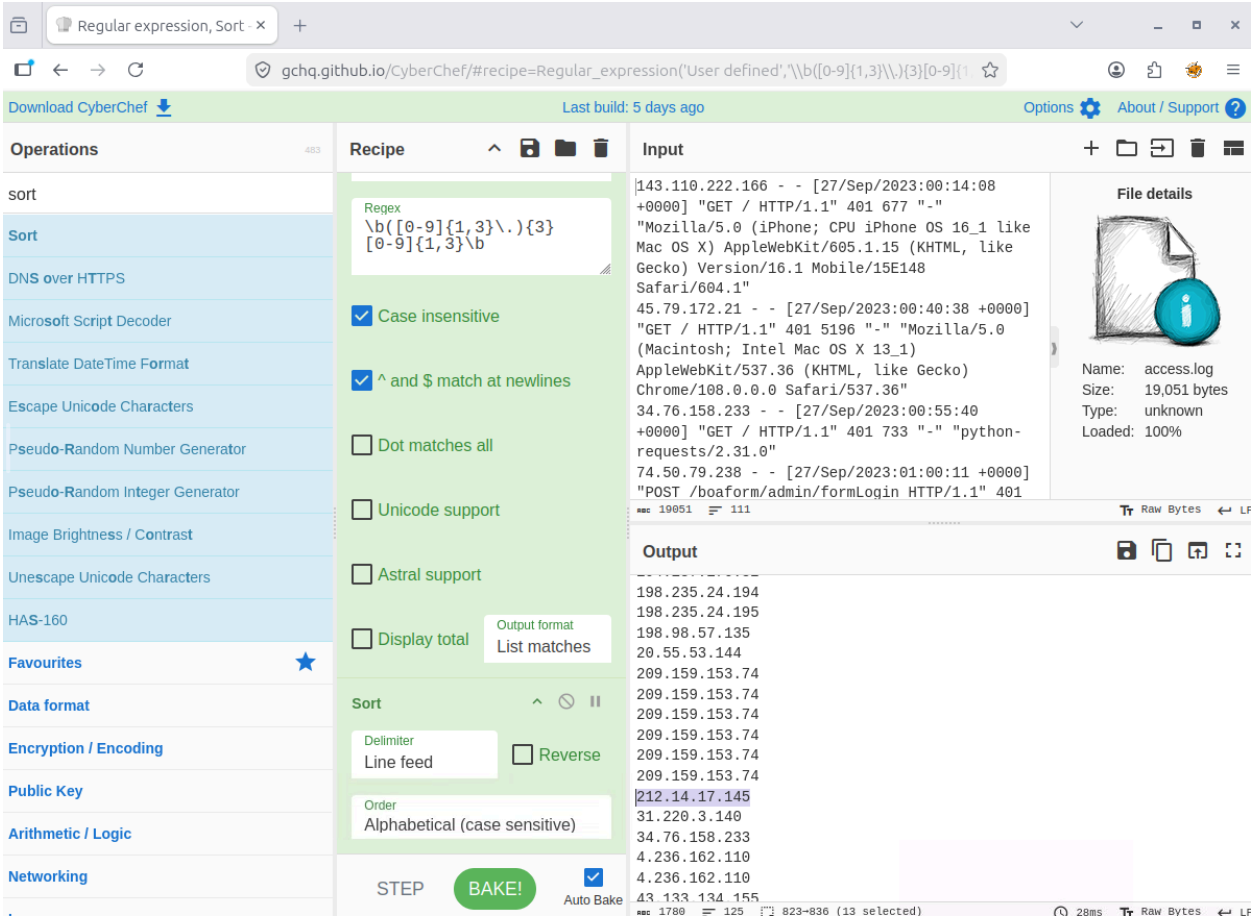
> How would you modify the original grep pattern above to match blog posts with an ID between 20-29? post=2[0-9]

> What is the name of the filter plugin used in Logstash to parse unstructured log data? Grok

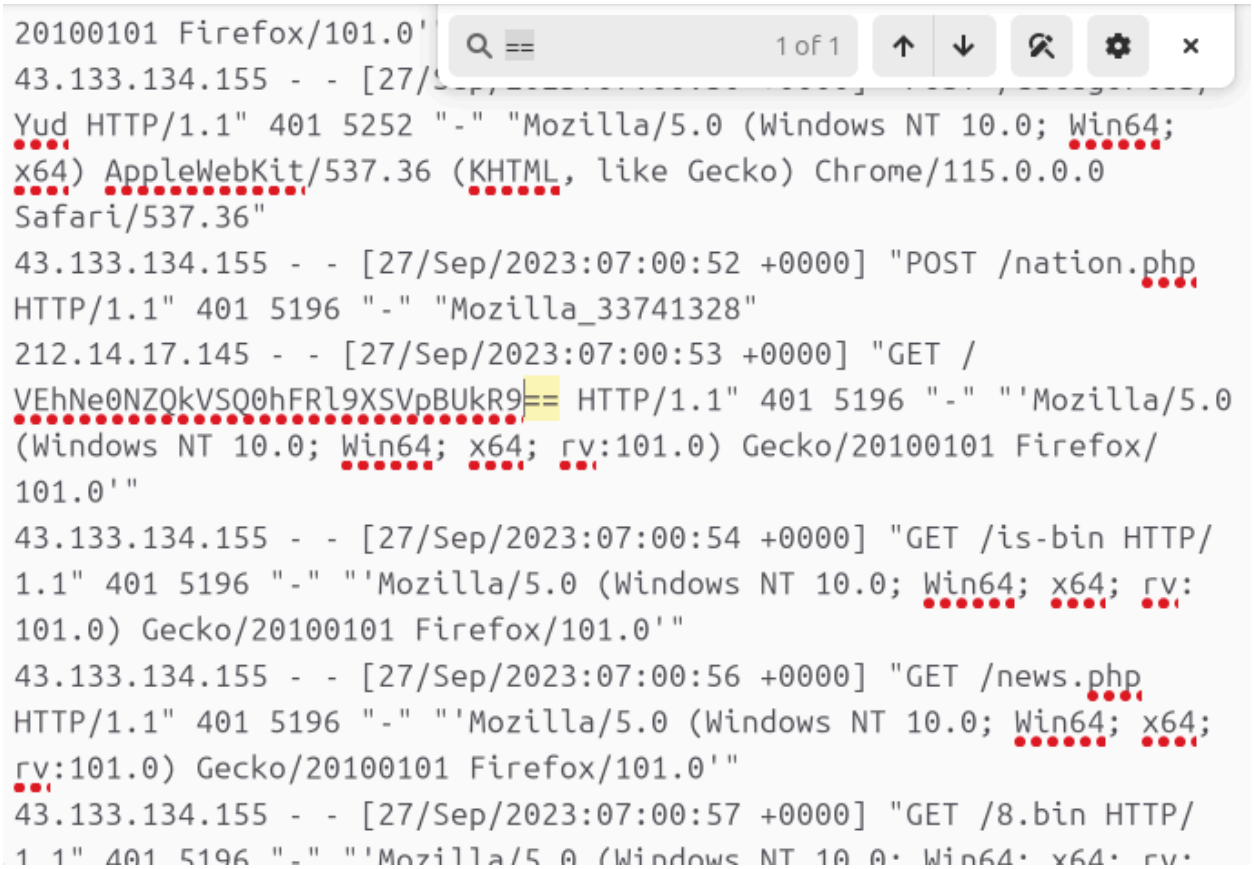
Task 8 - Log Analysis Tools: CyberChef

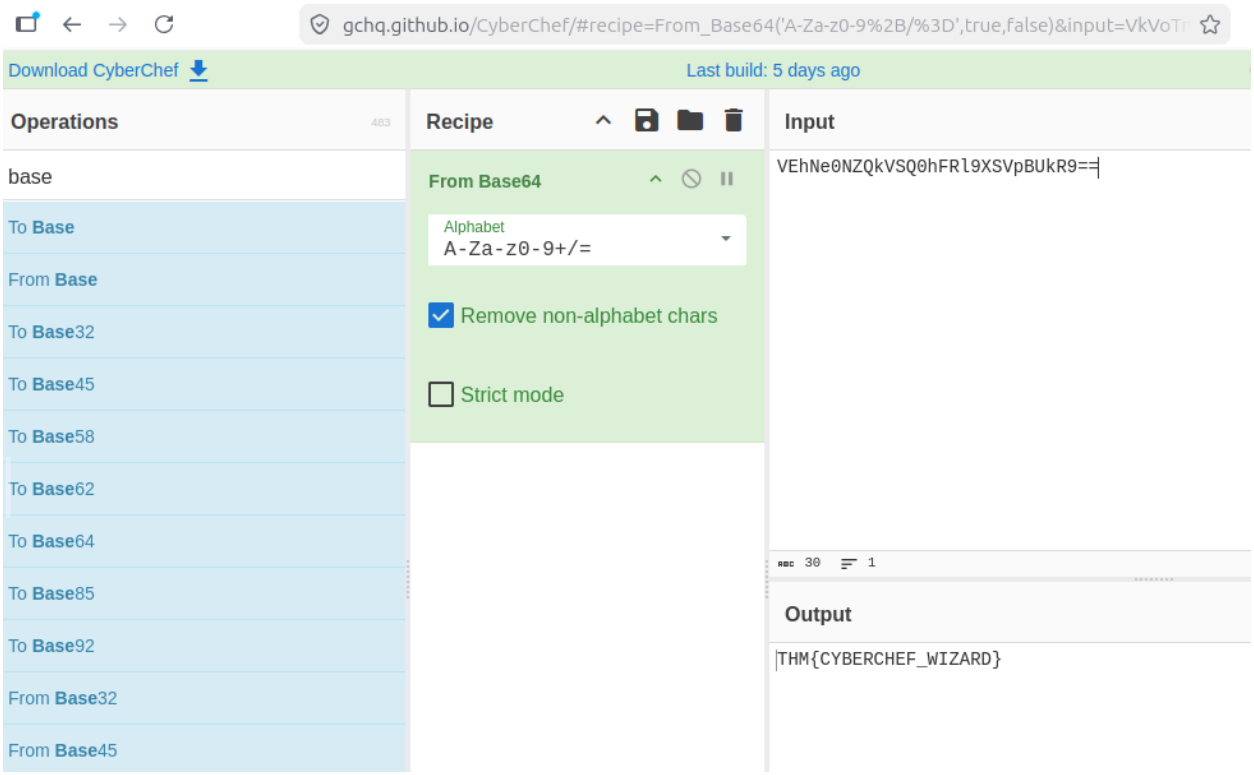
Answer the questions below

> Upload the log file named "access.log" to CyberChef. Use regex to list all of the IP addresses. What is the full IP address beginning in 212? 212.14.17.145

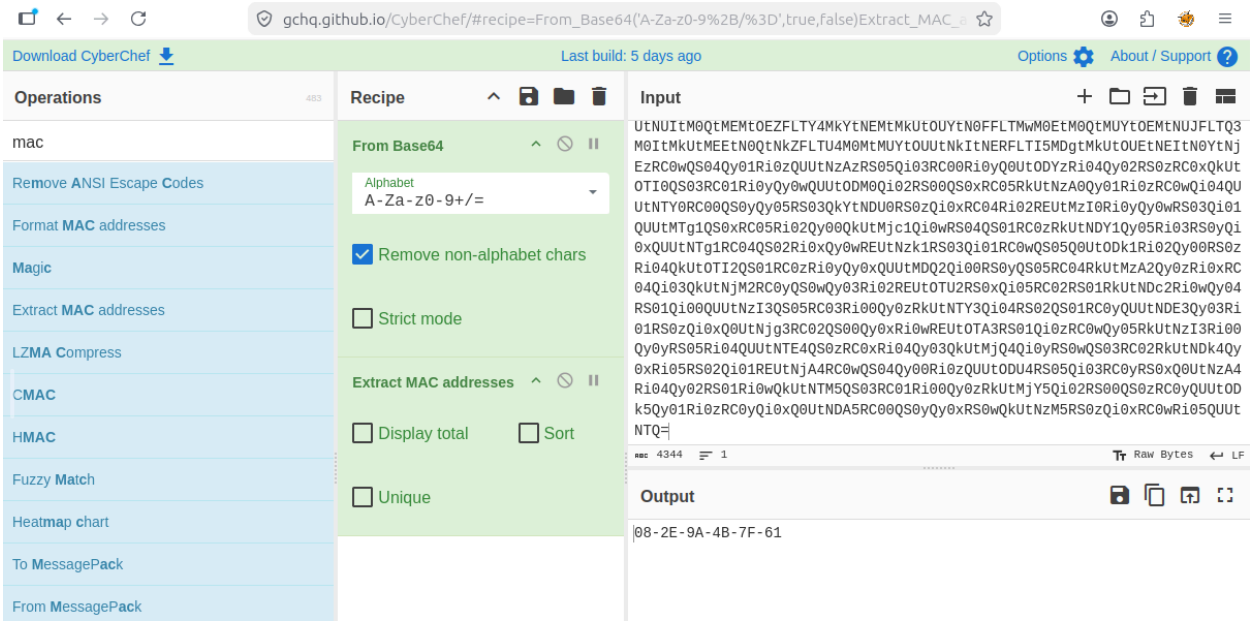


> Using the same log file from Question #2, a request was made that is encoded in base64. What is the decoded value? THM{CYBERCHEF_WIZARD}


20100101 Firefox/101.0'
==
1 of 1
43.133.134.155 - - [27/
Yud HTTP/1.1" 401 5252 "-" "Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.0.0
Safari/537.36"
43.133.134.155 - - [27/Sep/2023:07:00:52 +0000] "POST /nation.php
HTTP/1.1" 401 5196 "-" "Mozilla_33741328"
212.14.17.145 - - [27/Sep/2023:07:00:53 +0000] "GET /
VEhNe0NZQkVSQ0hFRl9XSVpBUkR9== HTTP/1.1" 401 5196 "-" "'Mozilla/5.0
(Windows NT 10.0; Win64; x64; rv:101.0) Gecko/20100101 Firefox/
101.0'"
43.133.134.155 - - [27/Sep/2023:07:00:54 +0000] "GET /is-bin HTTP/
1.1" 401 5196 "-" "'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:
101.0) Gecko/20100101 Firefox/101.0'"
43.133.134.155 - - [27/Sep/2023:07:00:56 +0000] "GET /news.php
HTTP/1.1" 401 5196 "-" "'Mozilla/5.0 (Windows NT 10.0; Win64; x64;
rv:101.0) Gecko/20100101 Firefox/101.0'"
43.133.134.155 - - [27/Sep/2023:07:00:57 +0000] "GET /8.bin HTTP/



> Using CyberChef, decode the file named "encodedflag.txt" and use regex to extract by MAC address. What is the extracted value? 08-2E-9A-4B-7F-61



Task 9 - Log Analysis Tools: Yara and Sigma

Sigma

Sigma(opens in new tab) is a highly flexible open-source tool that describes log events in a structured format. Sigma can be used to find entries in log files using pattern matching. Sigma is used to:

1. Detect events in log files
2. Create SIEM searches
3. Identify threats

Sigma uses the YAML syntax for its rules. This task will demonstrate Sigma being used to detect failed login events in SSH. Please note that writing a Sigma rule is out-of-scope for this room. However, let's break down an example Sigma rule

Sigma rule:

Key	Value	Description
title	Failed SSH Logins	This title outlines the purpose of the Sigma rule.
description	Searches sshd logs for failed SSH login attempts	This key provides a description that expands on the title.
status	experimental	This key explains the status of the rule. For example, "experimental" means that further testing or improvements must be done.
author	CMNatic	The person who wrote the rule.
logsource	product: linuxservice: sshd	Where can the log files that contain the data that we're looking for be found?
detection	sshd	This key lists what the Sigma rule is looking to find.
a0|contains	a0|contains: 'Failed'	In this case, look for all entries with "Failed".
a1|contains	a1|contains: 'Illegal'	In this case, look for all entries with "Illegal".
falsepositives	Users forgetting or mistyping their credentials	List cases where this entry may be present but doesn't necessarily indicate malicious behavior.

Yara

Yara(opens in new tab) is another pattern-matching tool that holds its place in an analyst's arsenal. Yara is a YAML-formatted tool that identifies information based on binary and textual patterns (such as hexadecimal and strings). While it is usually used in malware analysis, Yara is extremely effective in log analysis. Let's look at this example Yara rule called "IPFinder". This YARA rule uses regex to search for any IPV4 addresses. If the log file we are analyzing contains an IP address, Y

Yara rule:

Key	Example	Description
rule	IPFinder	This key names the rule.
meta	author	This key contains metadata. For example, in this case, it is the name of the rule's author.
strings	$ip = /([0-9]{1,3}\.){3} [0-9]{1,3}/ wide ascii	This key contains the values that YARA should look for. In this case, it is using REGEX to look for IPV4 addresses.
condition	$ip	If the variable $ip is detected, then the rule should trigger.

This YARA rule can be expanded to look for:

- Multiple IP addresses
- IP Addresses based on a range (for example, an ASN or a subnet)
- IP addresses in HEX
- If an IP address lists more than a certain amount (I.e., alert if an IP address is found five times)
- And combined with other rules. For example, if an IP address visits a specific page or does a certain action

Answer the questions below

> What languages does Sigma use? YAML

> What keyword is used to denote the "title" of a Sigma rule? title

> What keyword is used to denote the "name" of a rule in YARA? rule

Feature	Sigma	YARA
Purpose	Detect suspicious events in logs	Detect malware in files and memory
Data Source	SIEM logs, Windows events, network logs	Files, binaries, memory dumps
Format	YAML	YARA syntax
Used By	SOC Analysts, Detection Engineers	Malware Analysts, Threat Hunters
Detects	Attacker behavior	Malicious code/artifacts
Example	Failed logins, PowerShell abuse	Ransomware, trojans, malware families

YAML (YAML Ain't Markup Language) is a human-readable data serialization format used to store configuration files and structured data. It is popular because it is simple, easy to read, and uses indentation instead of brackets or tags.

> **Sigma detects malicious behavior in logs, while YARA detects malicious content in files and memory. Sigma is used for SIEM detections, whereas YARA is used for malware identification and classification.**